ProyectoPatrones

Plan técnico de trabajo colaborativo

Sistema POS web integrado con Odoo 20 Community
ASP.NET Core Web API · Frontend web · Clean Architecture · Patrones de Software

Equipo: Gabriel · Vaca · Lara

Documento de coordinación, alcance, ramas, validaciones, pruebas y criterios de aceptación

1. Propósito de este documento

Este documento transforma la especificación académica del proyecto integrador en un plan ejecutable para tres integrantes. Define qué debe entregar cada persona, cómo se conectan las entregas, qué debe existir en cada rama, cómo se revisan los cambios y cómo se valida el sistema antes de presentarlo.

La división cubre el alcance completo del PDF: POS web, API REST en ASP.NET Core, integración con Odoo 20 Community, Clean Architecture en frontend y backend, reglas de negocio, descuentos autorizados, ventas y pagos condicionados a la integración validada, historial, pruebas automatizadas, seguridad, documentación y defensa individual.

Las reglas adicionales de este plan —validaciones al teclear, límite de 40 caracteres, cédula de 10 números, búsqueda dinámica, búsquedas separadas por nombre e identificador, y paginación de 10 elementos— se consideran requisitos obligatorios del producto y se prueban junto con los requisitos del documento académico.

2. Alcance confirmado y límites

Área	Debe incluir	No se debe afirmar sin evidencia
POS	Catálogo, carrito, cantidades, descuentos autorizados, confirmación, método de pago, resultado e historial.	Que una venta o pago real funcionó si no existe respuesta verificable de Odoo.
Integración	Frontend → API .NET → Odoo 20 Community; API key solo en backend.	Acceso directo del navegador a Odoo o acceso de .NET a PostgreSQL para sustituir el POS.
Calidad	Clean Architecture, SOLID pertinente, Clean Code, Git, pruebas y README reproducible.	Usar patrones solo por cantidad o afirmar patrones no aplicados.
Odoo	Instancia local con PostgreSQL, módulo point_of_sale, productos, impuestos, POS y pago efectivo.	Modificar el código fuente de Odoo o desarrollar un módulo Python, salvo autorización expresa.
Operaciones condicionadas	RF-09 a RF-14 y P09 requieren comprobación temprana del mecanismo disponible.	Reemplazar integración real por simulador sin informar y justificar el bloqueo.

3. Arquitectura objetivo

```
Frontend (React/Vue/Angular/Vanilla justificado) → API REST ASP.NET Core → Odoo 20 Community + PostgreSQL
```

El frontend nunca accede directamente a Odoo ni a PostgreSQL. La API concentra autenticación, autorización, validaciones de aplicación, políticas de descuento, idempotencia, manejo de errores y el contrato de integración. Odoo conserva los datos empresariales y la operación POS; la solución solo mantiene datos propios estrictamente necesarios.

Capas reconocibles

Capa	Responsabilidad	Ejemplos
Domain	Entidades, valores, reglas puras y	Sale, SaleLine, DiscountPolicy,

	estados.	PaymentResult, PosSession.
Application	Casos de uso, puertos, DTOs y orquestación.	SearchProducts, CreateSale, GetHistory, GetSaleDetail.
Infrastructure	Adaptadores HTTP hacia Odoo, configuración, autenticación técnica y persistencia propia mínima.	OdooJson2Client, IdempotencyStore, OdooSettings.
Presentation	Endpoints REST, filtros HTTP, serialización y composición del frontend.	/api/products, /api/sales, pantallas POS.

Regla de dependencias: Domain no depende de frameworks ni de Odoo; Application depende de Domain y de interfaces; Infrastructure implementa interfaces; Presentation depende de Application. Las referencias deben apuntar hacia las políticas internas.

4. División del trabajo en tres frentes equivalentes

Frente A — Gabriel: Frontend POS, experiencia y validaciones

Responsabilidad: construir una interfaz completa y accesible que consuma exclusivamente la API .NET, mantenga la lógica visual separada de HTTP y permita demostrar el flujo completo del POS.

Entrega	Contenido obligatorio	Evidencia
Arquitectura frontend	Capas o módulos de presentación, aplicación y adaptadores HTTP; cliente API aislado; modelos/DTOs; estado del carrito separado de componentes visuales.	Árbol de carpetas, revisión de dependencias y README.
Catálogo	Consulta a /api/products; búsqueda dinámica por nombre y por código/ID; precio, disponibilidad e identificación.	Pantalla y prueba de respuesta real.
Carrito	Agregar, cambiar cantidades, eliminar, subtotales y total provisional; impedir cantidades inválidas.	P07 y demostración.
Descuentos	Mostrar solo políticas que la API marque autorizadas; reflejar restricciones y resultado; nunca decidir autorización solo en el navegador.	Pruebas de rechazo y cambio visual controlado.
Venta	Formulario de venta, método de pago configurado, confirmación, estado, identificador y errores comprensibles.	Flujo completo contra contrato de API.
Historial	Listado paginado de 10 elementos, filtros dinámicos, detalle de venta y estado real.	Paginación visible y consulta de detalle.
Validaciones UI	Bloqueo al escribir, paste y entrada programática; mensajes junto al campo; estados loading/empty/error.	Pruebas de teclado, pegado y mensajes.

Validaciones obligatorias del frontend

- Todos los campos de entrada tienen máximo 40 caracteres; el límite se aplica con maxlength y también en la lógica de entrada.

- Campos de solo letras: nombre, apellido, nombres usados en búsquedas, ciudad, país y cualquier campo que el formulario rotule “solo letras”. Se permiten letras Unicode y espacios donde corresponda; se bloquean números y símbolos.
- Campos de solo números: IDs numéricos, cantidades, importes cuando se introduzcan manualmente y cualquier campo que el formulario rotule “solo números”. Se bloquean letras, espacios y símbolos.
- Cédula: exactamente 10 dígitos; no se aceptan letras, espacios ni signos. La longitud se controla durante la escritura y la validación completa al enviar.
- Correo: máximo 40 caracteres, un solo símbolo @, parte local válida, dominio y una sola extensión final; debe aceptar ejemplos como usuario@gmail.com y rechazar usuario@gmail.com.com, usuario@@gmail.com, usuario gmail.com y dominios sin extensión.
- Fechas: el selector no permite días anteriores ni futuros; el backend vuelve a comprobar que la fecha sea la fecha actual del servidor.
- Búsquedas: cada entidad buscable debe tener un campo independiente por nombre y otro por ID/código cuando aplique. Los resultados se actualizan mientras se escribe y no se muestra todo el catálogo en una sola pantalla.
- Paginación: 10 registros por página; controles anterior/siguiente, página actual, total o rango; el filtro conserva su valor al cambiar de página.
- Errores: no ocultar silenciosamente respuestas 401, 409, 422, 502 o timeout; mostrar un mensaje entendible sin afirmar que la venta se guardó si Odoo no lo confirmó.

Criterio de terminado del Frente A

- No hay llamadas HTTP dispersas dentro de componentes visuales.
- El flujo demo funciona con respuestas reales de la API.
- Las pruebas de UI cubren validaciones, carrito, paginación, error de API y estado de confirmación.
- La interfaz puede explicar qué dato se está mostrando y si proviene de Odoo o de una operación local.

Frente B — Vaca: Backend .NET, casos de uso y reglas de negocio

Responsabilidad: implementar la API REST en ASP.NET Core con Clean Architecture, concentrar las reglas fuera de los controladores y entregar contratos estables para el frontend y los adaptadores de Odoo.

Entrega	Contenido obligatorio	Evidencia
Solución .NET	Projects o carpetas Domain, Application, Infrastructure y Presentation; composición por DI; configuración por opciones.	Diagrama de dependencias y build reproducible.
Autenticación	POST /api/auth/login; usuario operador, hash seguro, JWT o mecanismo definido; autorización de casos protegidos.	Pruebas de credenciales y 401/403.
Catálogo	GET /api/products?name=&code=&page=1&pageSize=10; contrato de productos consultados desde Odoo.	Paginación, filtros y pruebas.
Sesión POS	GET /api/pos/sessions/active; validar sesión abierta y datos necesarios para	P03 y evidencia de Odoo.

	vender.	
Venta	POST /api/sales; validar líneas, cantidades, precios de referencia, descuento, fecha, método de pago y sesión.	P01/P02/P04/P05/P06/P09.
Historial/detalle	GET /api/sales y GET /api/sales/{id}; respuestas paginadas y estados reales.	RF-13/RF-14 y pruebas.
Idempotencia	Idempotency-Key obligatorio o contrato equivalente; repetición devuelve el mismo resultado sin crear una segunda venta.	P06 y prueba de reenvío.
Errores	ProblemDetails consistente; timeouts, respuestas inciertas, errores de validación, autorización y Odoo sin éxito falso.	P05/P08 y ejemplos en README.
Configuración	Odoo URL, base, usuario, API key y timeouts por variables de entorno; jamás secretos en Git.	.env.example y escaneo de secretos.

Reglas de negocio que debe implementar el backend

- RN-01: rechazar una venta sin productos.
- RN-02: rechazar cantidad cero o negativa.
- RN-03: verificar precios contra Odoo; el frontend no puede imponer precios arbitrarios.
- RN-04: aceptar únicamente descuentos autorizados por la política vigente.
- RN-05: no confirmar una venta inconsistente.
- RN-06: una venta confirmada no vuelve a estado de preparación.
- RN-07: no informar pago exitoso sin confirmación del responsable.
- RN-08: una solicitud repetida no crea una segunda venta.
- La fecha de operación debe corresponder al día actual según el reloj del servidor y debe rechazarse cualquier fecha pasada o futura.
- La cédula, correo y campos generales se validan nuevamente en backend aunque el frontend ya los haya bloqueado.

Criterio de terminado del Frente B

- Los controladores son delgados y delegan en casos de uso.
- Las reglas pueden probarse sin levantar Odoo mediante dobles de prueba.
- Los contratos OpenAPI/Swagger están documentados.
- Los estados de éxito, rechazo, incertidumbre y repetición están diferenciados.
- El backend nunca expone la API key de Odoo al navegador.

Frente C — Lara: Integración Odoo, infraestructura, pruebas y calidad

Responsabilidad: preparar Odoo 20 Community, validar técnicamente el flujo real, implementar los adaptadores de infraestructura, coordinar pruebas de integración y consolidar la evidencia de patrones, seguridad y reproducibilidad.

Entrega	Contenido obligatorio	Evidencia
Odoo local	Docker/Compose con Odoo 20 Community y PostgreSQL; módulo point_of_sale; al menos dos productos	README, compose, capturas y datos de prueba.

	vendibles, precios, impuestos, POS y pago efectivo.	
Cuenta técnica	Usuario de integración con permisos apropiados y API key guardada solo en .env local/secret manager.	Checklist sin revelar la clave.
Sesión POS	Abrir sesión válida; registrar IDs reales de sesión, productos y método de pago.	Archivo de evidencia sin secretos.
Adaptador Odoo	Cliente HTTP en Infrastructure; JSON-2 y método pos.order.sync_from_ui(orders) como candidato a contrastar con la instancia; timeout, retries prudentes y trazas sanitizadas.	Petición, respuesta y registro observado.
Prueba temprana	Registrar durante semana 1 una venta real con dos productos y efectivo; si falla, documentar HTTP/trazas y notificar, sin simularla.	P09 reproducible.
Pruebas	Unitarias con doubles para Application; integración real Odoo; contrato HTTP; smoke test de frontend/backend.	Reporte ejecutable y resultados.
Patrones	Coordinar tres fichas con problema, alternativas, solución/categoría, consecuencias, clases y prueba.	docs/decisions/*.md.
CI/calidad	Build, tests, análisis, secret scan, Docker/variables y protección de rama.	Workflow y status checks.
Entrega	README final, diagrama, matriz requisitos-evidencias, guía de demo y checklist de defensa.	Pull request de release.

Pruebas mínimas asignadas al Frente C

ID	Comprobación	Resultado esperado
P01	Venta sin productos	Rechazada sin llamada de confirmación.
P02	Cantidad cero/negativa	Rechazada.
P03	Sesión POS cerrada o inválida	Rechazada con error de integración.
P04	Producto no habilitado	No se envía la venta.
P05	Timeout/error Odoo	No se declara éxito.
P06	Repetición/idempotencia	Mismo resultado o estado incierto controlado; nunca doble venta.
P07	Carrito/cantidades/total	Frontend actualizado correctamente.
P08	API caída	Mensaje comprensible.
P09	Venta real de dos productos y efectivo	Registro verificado en Odoo.
P10	Cambio de política/mecanismo	Cambio controlado sin modificar componentes no relacionados.

Criterio de terminado del Frente C

- Existe una instalación reproducible de Odoo.
- La integración real está probada o el bloqueo está documentado con evidencia para el docente.
- No hay secretos en Git.

- Las pruebas tienen comandos claros y resultados guardados.
- La documentación permite que otra persona ejecute el proyecto sin conocimiento privado del equipo.

5. Contratos REST comunes

Método	Ruta	Responsable principal	Regla
POST	/api/auth/login	Backend + Frontend	No devolver secretos; token solo tras credenciales válidas.
GET	/api/products?name=&code=&page=&pageSize=10	Backend + Odoo + Frontend	Máximo 10 por página; datos comerciales de Odoo.
GET	/api/pos/sessions/active	Backend + Odoo	No vender con sesión cerrada o inválida.
POST	/api/sales	Backend + Odoo	Idempotency-Key; no éxito sin confirmación.
GET	/api/sales?name=&id=&page=&pageSize=10	Backend + Frontend	Historial dinámico y paginado.
GET	/api/sales/{id}	Backend + Frontend	Detalle verificable de la operación.

Estas rutas son contratos internos propuestos, no endpoints oficiales de Odoo. Si el equipo los modifica, debe actualizar el README, OpenAPI, frontend, pruebas y evidencias.

6. Ramas, commits y Pull Requests

Ramas protegidas

Rama	Uso	Quién puede escribir
main	Código entregable y demostrable.	Nadie directamente; únicamente Pull Requests aprobados.
develop	Integración del trabajo terminado y validado.	Nadie directamente; únicamente Pull Requests aprobados.
feature/gabriel-frontend	Trabajo de Gabriel.	Gabriel mediante PR.
feature/vaca-backend	Trabajo de Vaca.	Vaca mediante PR.
feature/lara-odoo-quality	Trabajo de Lara.	Lara mediante PR.
release/v1.0	Preparación de entrega final.	Creada desde develop por PR.

Reglas obligatorias de Pull Request

- Toda modificación entra por Pull Request; no se permite push directo a main ni develop.
- Cada PR debe describir objetivo, archivos, requisitos cubiertos, pruebas ejecutadas, riesgos y evidencia.
- Se requieren mínimo 2 aprobaciones de integrantes distintos al autor antes del merge.
- Las aprobaciones deben mantenerse después de cambios nuevos; si cambia el código, se revisa nuevamente.

- Deben pasar los checks automatizados: build, tests, lint/format y secret scan.
- La rama debe estar actualizada con la rama destino antes del merge.
- El autor no aprueba su propio PR ni hace merge de su propio cambio.
- El merge recomendado es squash merge para conservar un historial legible.
- Si se detecta un conflicto, el autor resuelve el conflicto en su rama y vuelve a solicitar revisión.

Formato de ramas

```
feature/<integrante>-<tema>
fix/<integrante>-<tema>
chore/<integrante>-<tema>
docs/<integrante>-<tema>
```

Ejemplos: feature/gabriel-search-pagination, feature/vaca-idempotency, feature/lara-odoo-json2, test/lara-p09-integration, docs/gabriel-validation-matrix.

Formato de commit

```
feat(frontend): add dynamic product search
fix(api): reject repeated sales by idempotency key
test(odoo): verify cash sale with two products
docs(team): record discount policy decision
```

7. Protección de GitHub

El repositorio debe configurarse con reglas de protección para main y develop. La configuración mínima es: requerir Pull Request, exigir 2 aprobaciones, exigir que los checks pasen, descartar aprobaciones obsoletas cuando cambie el código, exigir rama actualizada, bloquear force-push y eliminar ramas fusionadas.

Para exigir específicamente que los dos revisores sean Vaca y Lara cuando el autor sea Gabriel, y las combinaciones equivalentes, GitHub necesita los nombres de usuario exactos o equipos configurados. La protección base de 2 aprobaciones se puede activar de inmediato; luego se completa CODEOWNERS con los handles reales del equipo.

```
docs/CODEOWNERS
# Sustituir los marcadores por handles reales de GitHub
/frontend/ @GabrielHandle @VacaHandle
/backend/ @VacaHandle @LaraHandle
/infrastructure/ @LaraHandle @GabrielHandle
/docs/ @GabrielHandle @VacaHandle @LaraHandle
```

8. Flujo semanal de cuatro semanas

Semana	Objetivo común	Evidencia
1	Requisitos, arquitectura, Odoo instalado, dos productos, POS, pago efectivo, primer GET de productos y prueba temprana P09.	Issue de arquitectura, captura/JSON sanitizado, PR inicial y decisión 1.
2	Catálogo, carrito, descuentos, RN-01–RN-05, contratos REST, pruebas unitarias iniciales.	P01/P02/P04/P07, PRs revisados.
3	Venta, pago, historial, detalle, idempotencia, errores, integración real	P03/P05/P06/P08/P09, trazas sanitizadas.

	y P09.	
4	Refactorización, tres decisiones de diseño, seguridad, README, demo y defensa individual.	P10, matriz de aceptación, release y ensayo.

9. Tres decisiones de diseño que deben documentarse

Decisión	Pregunta que debe responderse	Evidencia mínima
Integración externa Odoo	¿Cliente directo, gateway, adapter o servicio? ¿Cómo se manejan timeout y respuestas inciertas?	Ficha, clase adaptadora, test P05/P09.
Descuentos variables	¿Strategy/policy, reglas condicionales o configuración? ¿Cómo se agrega una política sin tocar carrito?	Ficha, código y test de política autorizada/no autorizada.
Idempotencia/estado	¿Cómo se evita duplicar una venta y cómo se distinguen confirmado, rechazado e incierto?	Ficha, contrato Idempotency-Key, P06 y demo.

Las fichas deben distinguir arquitectura, patrón, principio SOLID y práctica Clean Code. No se debe llamar “patrón” a toda carpeta ni usar Strategy, Adapter o State sin explicar el problema concreto que resuelve.

10. Checklist de entrega

- Repositorio Git con frontend, backend, pruebas, Docker/guía Odoo y configuración sin secretos.
- Clean Architecture reconocible en frontend y backend; dependencias revisadas.
- RF-01 a RF-17 trazados a código, prueba y evidencia.
- RN-01 a RN-08 cubiertos por reglas y pruebas.
- Validaciones de 40 caracteres, letras/números, cédula, correo y fecha actual implementadas en UI y backend.
- Búsquedas dinámicas por campos separados y paginación de 10 registros en productos, clientes/entidades aplicables, ventas e historial.
- Integración real o bloqueo técnico documentado con respuestas HTTP y trazas; nunca simulación presentada como real.
- P01–P10 ejecutables mediante comandos reproducibles.
- API key de Odoo fuera del repositorio; .env.example sin valores reales.
- README con requisitos, Docker, variables, arranque, pruebas, troubleshooting y demo.
- Tres decisiones de diseño con alternativas, consecuencias, ubicación y prueba.
- PR de release aprobado por dos integrantes distintos del autor y checks verdes.
- Cada integrante puede explicar y modificar una parte del código durante la defensa.

11. Guion de demostración

1. Levantar Odoo y PostgreSQL con Docker; comprobar productos, POS, impuestos, sesión y pago efectivo.

2. Iniciar la API .NET y comprobar Swagger.
3. Abrir frontend e iniciar sesión.
4. Consultar productos desde Odoo; probar búsqueda por nombre/código y paginación de 10.
5. Agregar dos productos al carrito, cambiar cantidades, aplicar un descuento autorizado y mostrar total.
6. Intentar una cantidad inválida y un descuento no autorizado; mostrar mensajes.
7. Confirmar una venta con Idempotency-Key; comprobar identificador, estado y respuesta real.
8. Repetir la misma solicitud; demostrar que no se duplica.
9. Abrir historial, navegar páginas de 10 y consultar detalle.
10. Mostrar evidencia P09 en Odoo y explicar cualquier limitación reproducible.
11. Revisar un PR, las dos aprobaciones y los checks de GitHub.
12. Responder un cambio pequeño: por ejemplo, agregar una política de descuento sin modificar carrito ni adaptador Odoo.

12. Nota de seguridad y honestidad técnica

La API key, contraseñas y tokens nunca deben entrar al repositorio, capturas públicas o frontend. Las evidencias deben ocultar valores sensibles. Si Odoo responde con error, timeout o resultado incierto, el sistema debe comunicar “operación no confirmada” y conservar información técnica suficiente para investigar. La calificación valora la comprensión y la evidencia; no se debe presentar un mock como una venta real.

Fin del plan de trabajo